Event Sourcing & CQRS

eason: 4 · Enterprise · Category: Enterprise · Level: Intermediate · Build with Waqa

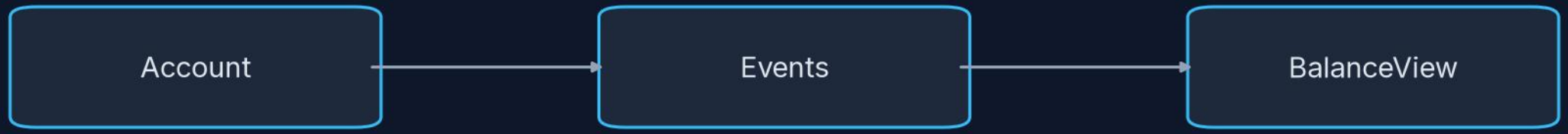
Today's Hook

- Today: Event Sourcing & CQRS as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Event Sourcing & CQRS structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Event Sourcing & CQRS
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/enterprise/eventsourcing/Account.java
package com.buildwithwaqar.patterns.enterprise.eventsourcing;

import java.util.ArrayList;
import java.util.List;

sealed interface AccountEvent permits Deposited, Withdrawn {}
record Deposited(int amount) implements AccountEvent {}
record Withdrawn(int amount) implements AccountEvent {}

/** Minimal event-sourced aggregate + separate read model (CQRS). */
public final class Account {
    private final List<AccountEvent> events = new ArrayList<>();
    private int balance;

    public void deposit(int amount) {
        apply(new Deposited(amount));
    }

    public void withdraw(int amount) {
        if (amount > balance) throw new IllegalStateException("insufficient");
        apply(new Withdrawn(amount));
    }
```

Implementation (2)

```
    private void apply(AccountEvent e) {
        switch (e) {
            case Deposited(var a) -> balance += a;
            case Withdrawn(var a) -> balance -= a;
        }
        events.add(e);
    }

    public int balance() { return balance; }
    public List<AccountEvent> events() { return List.copyOf(events); }
}

/** Read model projected from events. */
final class BalanceView {
    private int balance;
    void project(AccountEvent e) {
        switch (e) {
            case Deposited(var a) -> balance += a;
            case Withdrawn(var a) -> balance -= a;
        }
    }
```

Implementation (3)

```
    int balance() { return balance; }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.enterprise.eventsourcing;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Event Sourcing / CQRS walkthrough ===");
    var account = new Account();
    account.deposit(100);
    account.withdraw(40);
    System.out.println("writeModelBalance=" + account.balance());
    System.out.println("events=" + account.events());
    var view = new BalanceView();
    account.events().forEach(view::project);
    System.out.println("readModelBalance=" + view.balance());
  }
}
```

Walkthrough Output

```
=== Event Sourcing / CQRS walkthrough ===
writeModelBalance=60
events=[Deposited[amount=100], Withdrawn[amount=40]]
readModelBalance=60
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Event Sourcing & CQRS
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Event Sourcing & CQRS → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.